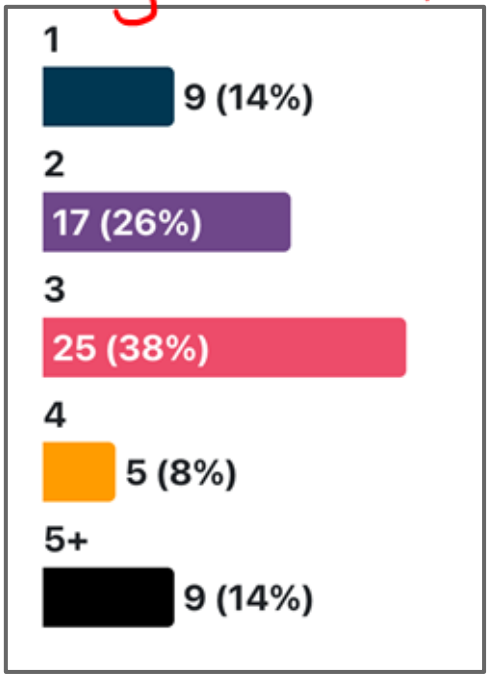
Try and replicate these results by voting




clicker.cs.illinois.edu

Q1

~Code~
340

# CS 340

Internet Big Picture
and APIs

OR

How many boba tea places did you pass on the way to class

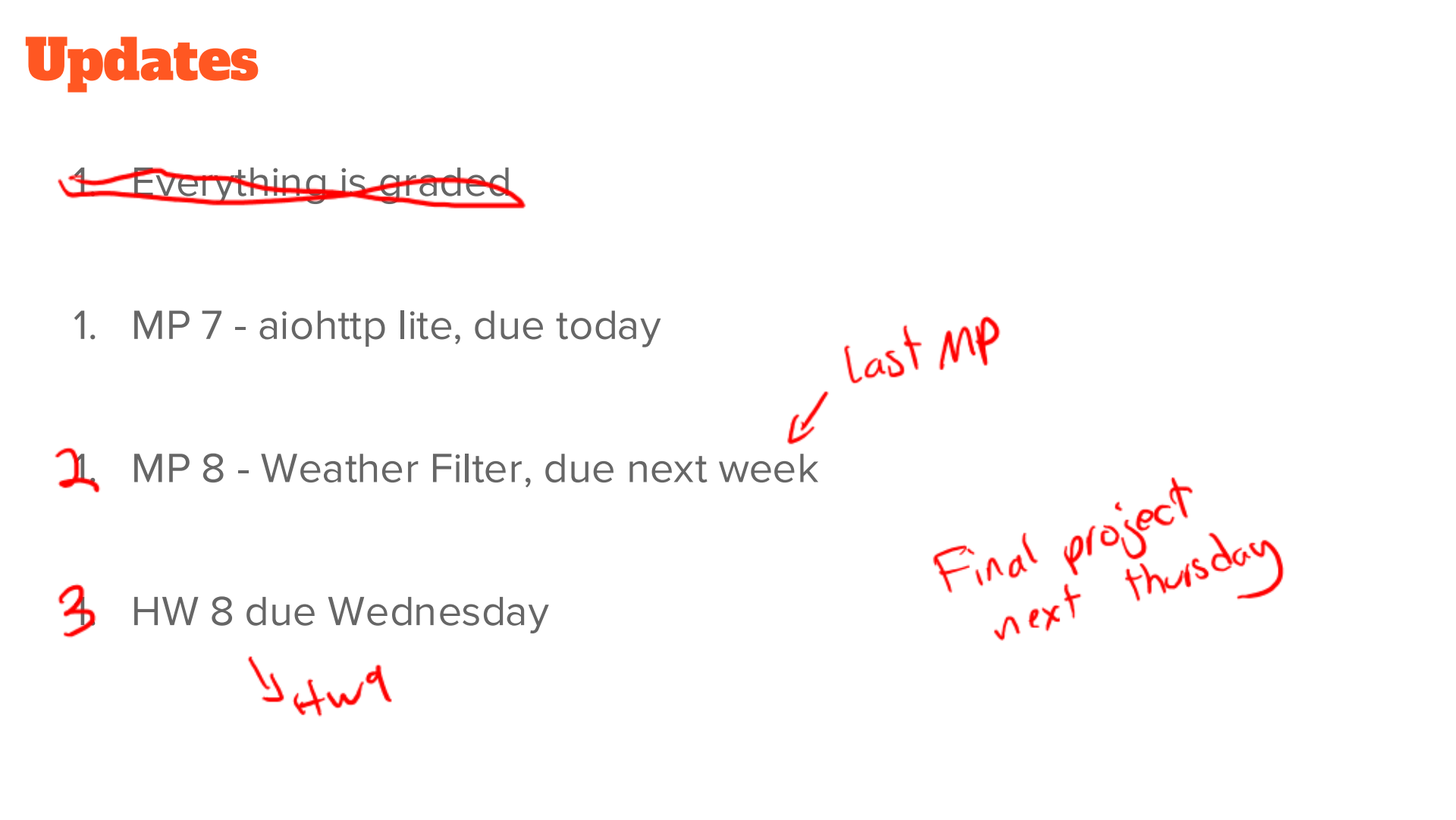
# Updates

1. ~~Everything is graded~~

1. MP 7 - aiohttp lite, due today

2. MP 8 - Weather Filter, due next week

3. HW 8 due Wednesday



# Internet Big Picture and APIs

**Today's LGs -** Build up your mental model of how the internet, IP, TCP, UDP, client-server, and APIs all fit together.

Be able to answers questions about what parts of a internet packet do.
Be able to describe the trade-off between TCP and UDP
Be able to write and use an API

# Agenda

1. Python Function Arguments
2. Internet Big Picture
    a. TCP vs. UDP
    b. Communication Models
3. webAPI

# Python - function arguments

positional

keyword

```
1  def greet(name, greeting="Hello", punctuation="!"):
2      print(greeting + " " + name + punctuation)
3
4  greet("Jule")
5  greet("Jule", punctuation="?")
6
```

input

```
Hello Jule!
Hello Jule?
```

# Python - function arguments

```
1  def greet(name, greeting="Hello", punctuation="!"):
2      print(greeting + " " + name + punctuation)
3
4  greet("Jule", greeting="Hi")
5  greet("Jule", greeting="Heyo", punctuation="...")
6
```

input

```
Hi Jule!
Heyo Jule...
```

# Python - *args

positional arguments

args is of type tuple

ordered

immutable

```
1  def add(*args):
2      print(args)
3      print(*args)
4      sum_ = 0
5      for item in args:
6          sum_ += item
7      return sum_
8
9  print(add(3, 1, 5, 6))
10
```

```
(3, 1, 5, 6)
3 1 5 6
15
```

add(2,5)

# Python - **kwargs

Keywords → dictionary

```
1  def func(**kwargs):
2      print(kwargs)
3      if "name" in kwargs:
4          print(kwargs["name"])
5      else:
6          print("Nope")
7
8  func(name="Jule", num=55)
9  func(potato="no")
10
```

```
{'name': 'Jule', 'num': 55}
Jule
{'potato': 'no'}
Nope
```

# Python - *args and **kwargs

```
1  def func(*args, **kwargs):
2      if "name" in kwargs:
3          print(kwargs["name"])
4      else:
5          if len(args) > 0:
6              print(args[0])
7
8  func(6, "hi", name="Jule", num=55)
9  func("hi", other="Jule")
10 func(64)
11 func()
```

```
Jule
hi
64
```

# Python - *args and **kwargs

```
1  def func(*args, **kwargs):
2      if "name" in kwargs:
3          print(kwargs["name"])
4      else:
5          if len(args) > 0:
6              print(args[0])
7
8  def pass_it(*args, **kwargs):
9      func(*args, **kwargs)
10
11 pass_it(6, "hi", name="Jule", num=55)
12 pass_it("hi", other="Jule")
13
14
```

positional

keyword

```
Jule
hi
```

# What prints?

```
1  def func(*args, **kwargs):
2      if "name" in kwargs:
3          print("yay")
4      else:
5          print("no")
6
7  def func2(*args, **kwargs):
8      func(*args)
9
10 func2(6, "hi", name="Jule", num=55)
```

positional

keyword

func(6, "hi")

no

clicker.cs.illinois.edu

Q2

~Code~

340

# What does the IP layer provide?

clicker.cs.illinois.edu

Q3

~Code~
340

A. Information of what computer the packet needs to go to.

B. Information of what port the packet needs to go to.

C. Information of what process the packet needs to go to.

D. Information of how the payload in the packet is formatted.

# What does the transport (TCP/UDP) layer provide?

clicker.cs.illinois.edu

Q4

~Code~
340

A. Information of what computer the packet needs to go to.

B. Information of what port the packet needs to go to.

C. Information of what process the packet needs to go to.

D. Information of how the payload in the packet is formatted.

# What does the application (HTTP) layer provide?

clicker.cs.illinois.edu

Q5

~Code~
340

A. Information of what computer the packet needs to go to.

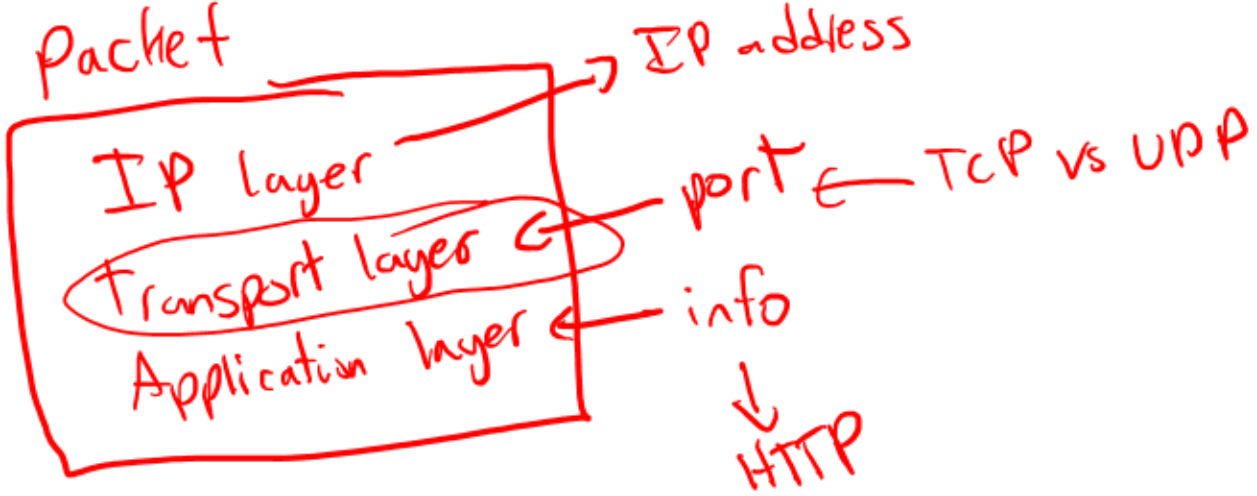
B. Information of what port the packet needs to go to.

C. Information of what process the packet needs to go to.

D. Information of how the payload in the packet is formatted.

# The Internet

A global system of computers that send packets of information



# Reliability Vs. Speed Tradeoff

TCP - guarantees arrival and order of packets

- more setup and ~~more~~ bigger headers
- Great for big things that span multiple packets

UDP - guarantees nothing

fast, less overhead

- great for small things we can send again
- great for speed

# Example: Multiplayer Racing Game



# When building a racing game, which packets are better suited to use TCP? The options not selected would use UDP.

clicker.cs.illinois.edu

Q6

~Code~
340

A. Packets relaying information of where certain players are (player position) UDP speed

B. Packets relaying information of if a player got a power up TCP

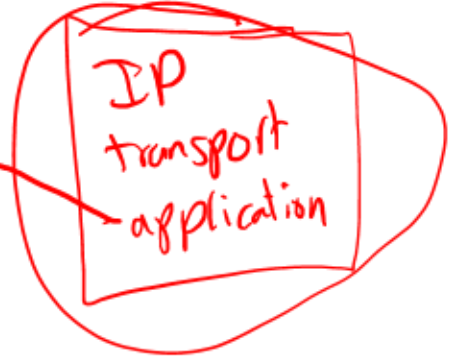
C. Packets relaying information of when a player passes the finish line TCP



# Big Ideas

- Information is sent on the internet... through packets
- Packets aren't guaranteed... to arrive in any order or at all
- Different kinds of information have different needs for...
  speed and reliability
- TCP and UDP are two transport protocols that handle these tradeoffs.
  - TCP - guarantees delivery and order but slow
  - UDP - is fast, but guarantees nothing

# Internet Communication Models




- Client-Server
  - responds
  - initiates requests
  - example - website
- Peer-to-peer - back and forth, no central server
  - blockchain
- Mix - central server handles coordination the individual devices just do peer-to-peer
  - example - facetime

# Client-Server Example: Websites

HTTP- protocol used

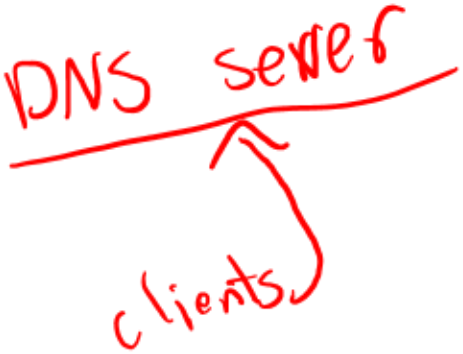





# Client-Server Example: DNS

Domain name system

google.com → IP address







UDP

↓

small and

we can resend

# Does HTTP use TCP or UDP?

not true DNS UDP

A. ~~TCP -~ it has to for the client-server model

B. TCP - HTTP assumes reliability to work

C. ~~UDP~~ - HTTP doesn't assume reliability and prefers speed

D. UDP - is can't use TCP for the client-server model

clicker.cs.illinois.edu

Q7

~Code~

340

# Does DNS typically use TCP or UDP?

clicker.cs.illinois.edu

Q8

~Code~
340

A. TCP - it has to for the client-server model
B. TCP - DNS assumes reliability to work
C. UDP - DNS is small enough to fit in a packet and prefers speed
D. UDP - is can't use TCP for the client-server model

# API

- application programming interface

Defined way for two pieces of software to talk to each other.

# Web API - sever

↳ over the internet

usually uses HTTP

↓

TCP

↓

client-sever model

# Web API Summary

Defines:

- what you can ask for
- How to ask for it
- What you get back

end point

GET /fact

implemented as a server described in documentation

usually gives back json

# API Example

```python
from aiohttp import web
import random

routes = web.RouteTableDef()

facts = [
    "The sky is blue",
    "The grass is green",
]

@routes.get("/fact")
async def get_fact(request: web.Request) -> web.Response:
    fact = random.choice(facts)
    return web.Response(text=fact)
```

end point GET /fact

Docs

Get

/fact

↓ returns string of fun fact

# Working with an API

A) Use curl commands in the terminal to access API endpoints

mp8

B) Use code to access API endpoints

# API Example - Accessing the API endpoint

```python
facts = [
    "The sky is blue",
    "The grass is green",
]

@routes.get("/fact")
async def get_fact(request: web.Request) -> web.Response:
    fact = random.choice(facts)
    return web.Response(text=fact)
```

curl -X GET website/fact

local → 127.0.0.1:8080

or funfacts.com

↖ return a string

# API Example - Sending info to the API endpoint

```python
@routes.post("/fact")
async def add_fact(request: web.Request) -> web.Response:
    data = await request.json()
    fact = data["fact"]

    facts.append(fact)

    return web.json_response({
        "message": "Fact added!",
        "fact": fact
    }, status=201)
```

```
curl -X POST website/fact -H "Content-Type: application/json" -
d '{"fact": "Cats sleep 16 hours a day"}'
```

# API Example - Sending info to the API endpoint

```
@routes.get("/fact")
async def get_fact(request: web.Request) -> web.Response:
    index = int(request.query.get("index", 0))
    fact = facts[index % len(facts)]

    return web.json_response({"fact": fact})
```

curl -X GET websites/fact?index=1"

Query parameters

? key=value & key2=value2...

# API Example - Accessing the API endpoint

server #1

```python
from aiohttp import web, ClientSession

routes = web.RouteTableDef()

@routes.get("/get-fact-from-api")
async def get_fact_from_api(request: web.Request) -> web.Response:
    async with ClientSession() as session:
        async with session.get("http://127.0.0.1:8080/fact") as resp:
            data = await resp.json()

    return web.json_response({
        "source": "external API",
        "data": data
    })
```

server #2

```python
@routes.post("/fact")
async def add_fact(request: web.Request) -> web.Response:
    data = await request.json()
    fact = data["fact"]

    facts.append(fact)

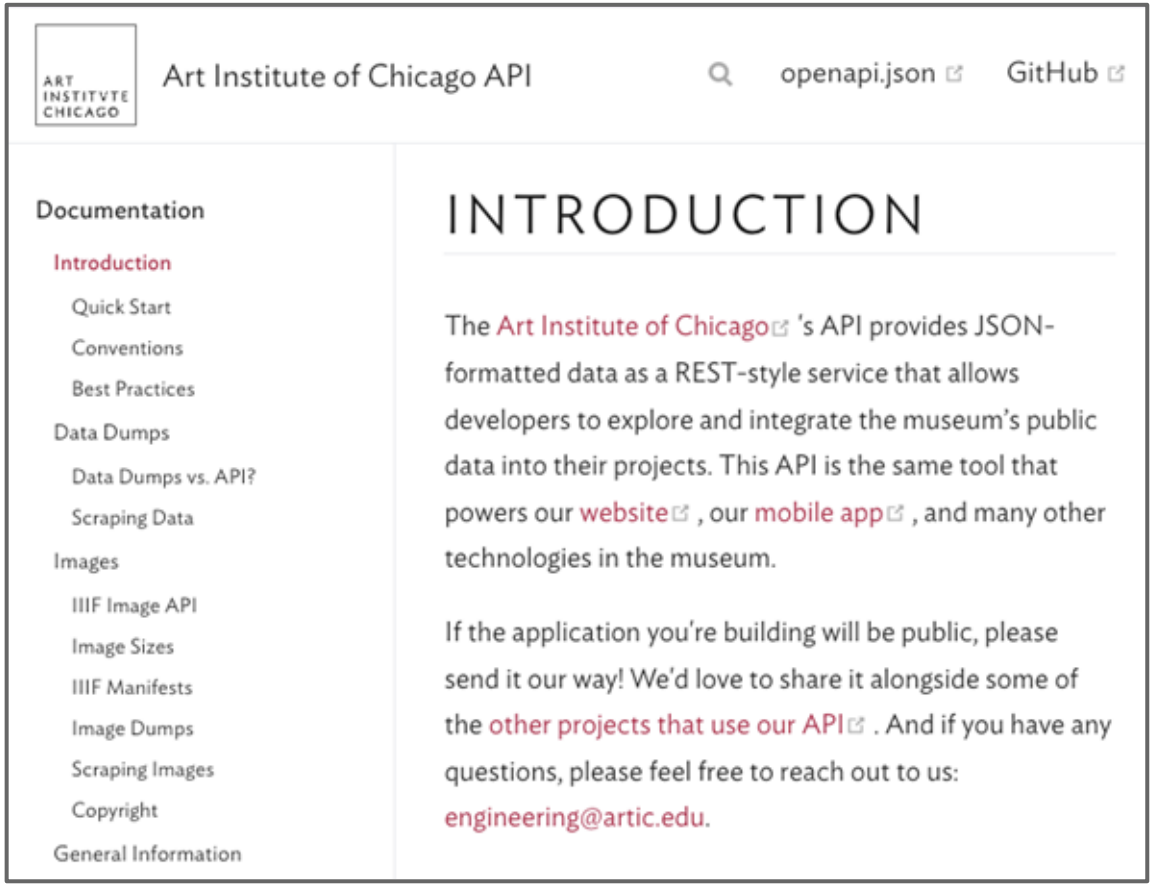
    return web.json_response({
        "message": "Fact added!",
        "fact": fact
    }, status=201)
```

client

fact()

- server

# API Example - Art Institute of Chicago API



# API Example - Art Institute of Chicago API

`GET /artworks`

A list of all artworks sorted by last updated date in descending order. For a description of all the fields included with this response, see here.

AVAILABLE PARAMETERS:

- `ids` - A comma-separated list of resource ids to retrieve
- `limit` - The number of resources to return per page
- `page` - The page of resources to retrieve
- `fields` - A comma-separated list of fields to return per resource
- `include` - A comma-separated list of subresource to embed in the returned resources.

  Available options are:
  - `artist_pivots`
  - `dates`
  - `place_pivots`
  - `sites`

# API Example - Art Institute of Chicago API

```
curl "https://api.artic.edu/api/v1/artworks?limit=10&fields=id,title"
```

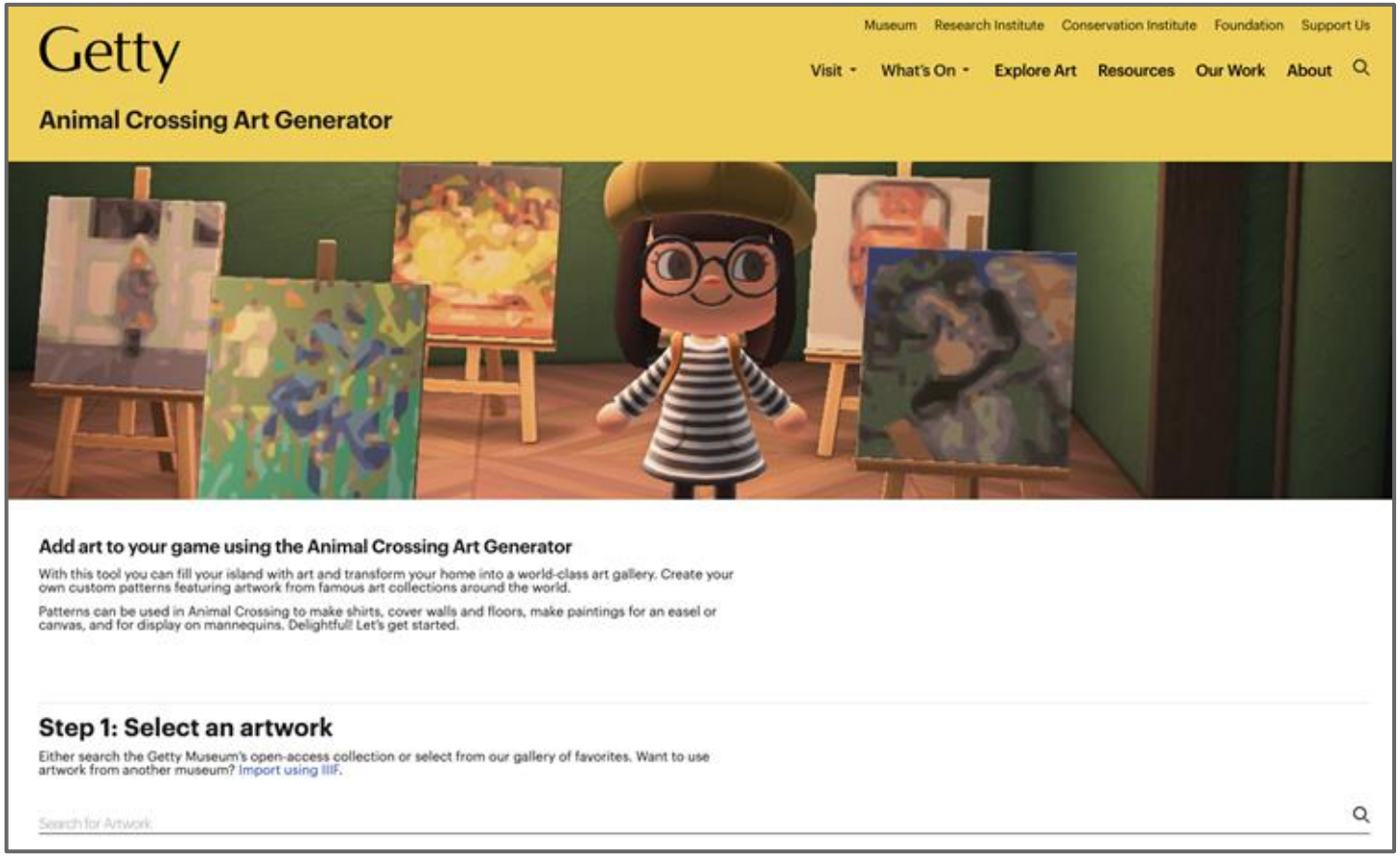
# API Example - Art Institute of Chicago API

```python
@routes.get("/monet")
async def get_monet(request: web.Request) -> web.Response:
    async with ClientSession() as session:
        async with session.get(
            "https://api.artic.edu/api/v1/artworks/search",
            params={
                "q": "Monet",
                "limit": 5
            }
        ) as resp:
            data = await resp.json()

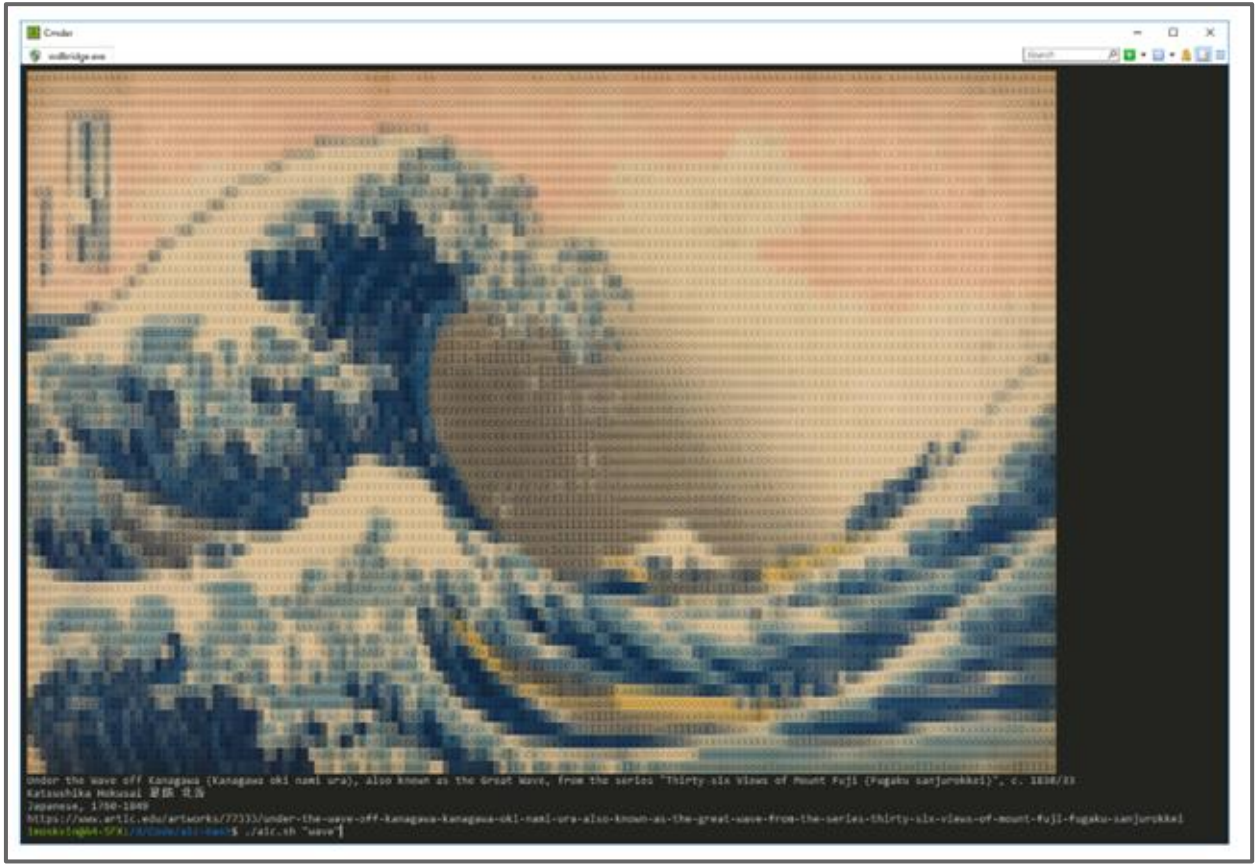
    artworks = [art["title"] for art in data["data"]]

    return web.json_response({
        "artist": "Monet",
        "artworks": artworks
    })
```

# API Example - Art Institute of Chicago API



# API Example - Art Institute of Chicago API



# Using a web API

A) APIs have rate limits - error 429 – too many requests from a client within a time limit.

a) Why? - protects servers from overloading and abuse

a) What to do?

- get a lot of data once
- caching
- delay

# MP 8 - Weather filter

write a web server that calls the national weather service API and add a filter to an image